Network Paths
Host -> Access Network -> Access ISP -> (Regional ISP) -> (IXP/Peering) -> (Global ISP) -> repeat down -> Host

() -> optional, IXP: Internet exchange point

Host A (your laptop)
→ Home router
→ Residential Access Network
→ Access ISP (e.g., Singtel, StarHub)
→ **Regional ISP (Southeast Asia transit)**
→ **IXP (international peering point)**
→ **Global ISP backbone (submarine cable)**
→ **Regional ISP (Japan)**
→ **Access ISP (Japan ISP)**
→ IXP (Internet Exchange Point) (local peering)
→ Content Provider Network (CDN edge node)
→ Server (Host B)

If not overseas, then local connection will skip regional and global

Who Runs the Internet?
Network Information Centre (NIC) administers IP address & Internet Naming
The Internet Society (ISOC) - Provides leadership in Internet related standards, education, and policy around the world.
The Internet Architecture Board (IAB) - Authority to issue and update technical standards regarding Internet protocols.
Internet Engineering Task Force (IETF) - Protocol engineering, development and standardization arm of the IAB. Internet standards are published as RFCs (Request For Comments)

Tool	Purpose	Usage
ping	- check host reachable- measure RTT- packet loss - uses ICMP Echo Request / Echo Reply (Layer 3)	ping google.com -c <count> (Linux/macOS) -n <count> (Windows)
traceroute / tracert	- show path to host- list hops (routers)- per-hop delay - uses TTL expiry + ICMP Time Exceeded (UDP probes on Linux default, ICMP on Windows)	Windows: tracert google.com Linux/macOS: traceroute google.com
mtr	- ping + traceroute combined - live hop stats, loss per hop - uses ICMP or UDP probes	Linux/macOS: mtr google.com Windows: WinMTR
nslookup	- basic DNS query- domain ↔ IP - uses DNS protocol over UDP (sometimes TCP for large replies)	nslookup google.com
dig	- advanced DNS query - full DNS response - uses DNS over UDP (TCP fallback)	dig google.com
telnet	- test TCP connection to port - check if service listening - uses raw TCP (no encryption)	telnet google.com 80
nc (netcat)	- test TCP/UDP ports - scan ports - simple client/server	nc -vz host port (TCP) nc -vu host port (UDP)
curl	- HTTP/HTTPS requests - API testing - uses HTTP over TCP	curl https://google.com
Wireshark	- capture packets - deep protocol inspection	GUI app (all OS)
tcpdump	- CLI packet capture - traffic filtering - captures raw Ethernet/IP/TCP/UDP packets	tcpdump -i interface
ipconfig / ip / ifconfig	- show IP config - interfaces, gateway, DNS	Windows: ipconfig /all Linux: ip addr macOS: ifconfig
netstat / ss	- show active connections, open ports - TCP/UDP socket states	Windows/macOS: netstat -an Linux: ss -tulnp
nmap	- port scan, service detection - OS fingerprinting - sends TCP SYN / UDP probes / ICMP discovery packets	nmap target

CRC is Binary modulo 2 division, xor generator with D + R and if 0 at the end no corruption, used at link layer due to burst error nature; upper layers use lighter checksums
For UDP: 1s Checksum, Calculation is add all, then flip, verification is add all = 1s all
Error free data rate, C = B * $\log_2$(1+SNR), B is frequency band, SNR is signal to noise ratio

Link layer Access method is **hard-coded/protocol-defined per technology**
CSMA waiting time = **queueing delay** (But depends on context)

If Links are all reliable, TCP still needed to handle reordering due to changing routes from packet switching/ routing loops/ hardware failures
Link layer = hop-by-hop reliability
TCP = end-to-end reliability

Private IP ranges: 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16

Protocol	Pros	Cons
Slotted ALOHA	- very simple - no central control	- high collision rate - max efficiency ~37% - needs time sync
Pure ALOHA	- simpler than slotted - no sync needed	- very inefficient (~18%) - high collision prob
CSMA/CD	- efficient vs ALOHA - simple distributed control - good for wired LANs	- collision waste still exists - poor under high load - not usable in wireless
TDMA (TIme division)	- no collisions - predictable delay - fair access	- wasted slots if idle - needs tight synchronization - inflexible
FDMA (extra)	- no collisions - continuous transmission possible	- inefficient for burst traffic - fixed bandwidth allocation
Token Passing	- no collisions- fair access- stable under high load	- token overhead- token loss recovery needed- delay increases with many nodes
Polling	- simple logic- no collisions- good for centralized systems	- polling overhead- single point of failure- latency if many idle nodes

Two hosts A and B are connected via a router. The link rate is 1 Mbps and propagation delay is 40 ms per link. The maximum size of a packet is 1 Kb and packet header is 80 bits Suppose sender sends as much data as possible in a packet, packets are sent continuously and no packet is corrupted or lost during transmission
How long (in milliseconds) does it take to send a 400 Kb file from A to B (from when the first bit of the first packet leaves A to when last bit of the last packet arrives at B)?
#of pkt = Ceil(400*10^3 / (1000-80)) = 435
Total # of bits sent = 435*80 + 400,000 = 434800
Length of first 434 packets: 1000
Length of last packet: 800
End-to-end delay = 1000/10^3 + 434800/20^3 + 40 + 40 = 515.8 ms

Public key cryptography uses both public and private keys. Let Alice's public key be K^+_A and private key be K^-_A, Bob's public key be K^+_B and private key be K^-_B Alice sends a message m to Bob. Describe how they can ensure message confidentiality and message authenticity using only these 4 keys.
1. Alice encrypts m with her private key to create digital signature $K^-_A(m)$.
2. Alice concatenates message with digital signature m || $K^-_A(m)$, and encrypt the extended message with Bob's public key: $K^+_B(m \,||\, K^-_A(m))$, sending it to Bob.
4. Bob decrypts the received message using his private key
5. Bob then uses Alice's public key to decrypt digital signature and verify
6. If m = m', message authenticity (and integrity) are preserved.

Suppose X sets a reasonable timeout value for this TCP connection. Retransmission is always successful. Estimate the timeout value that X chooses. Justify your answer and state clearly any assumptions you make.
Assumption: packets may be lost or corrupted but will not be reordered by the network.
The previous packet C is received at 110 ms. Once corresponding ACK reaches X, X will start a timer for packet D. When timer expires, D will be resent and received by Y at 190 ms.
Assume propagation delay is d ms. ACK of packet C take d to reach X. Timeout period is (slightly greater than) 2d. Retransmission takes another d. Therefore 4d = 190 - 110 Timeout value chosen by X is slightly greater than 2d which is 40 ms.

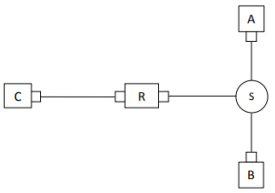
ICMP errors do NOT trigger ICMP error responses again (RFC rule: no ICMP-in-response-to-ICMP-error)

TCP Congestion Control:
Slow start (cwnd < ssthresh, slow start threshold**):** cwnd (congestion window) starts small and doubles every RTT (1 → 2 → 4 → 8 → …) to quickly probe available bandwidth
After slow start (cwnd >= ssthresh): Grow linearly, cwnd += 1 per RTT

3 duplicate ACKs: indicates mild loss → ssthresh = cwnd/2, cwnd = ssthresh (halve rate, continue sending)
Timeout: indicates severe congestion → ssthresh = cwnd/2, cwnd = 1 MSS (reset to very low rate and restart slow start)

Delayed ACKS sent when:
- Receiving 1 pkt and 500ms has passed
- Receiving 2 pkts.
- Receiving an Out of order packet
- Receiving a pkt that fills in smth that was out of order before.

Name	What it is + limits it	What happens if exceeded
Application payload	data generated by app before transport; limited by TCP segmentation (MSS/window)	split into multiple TCP segments
TCP segment / MSS	MSS = max TCP data per segment; limited by MTU (IP + TCP headers)	TCP splits into smaller segments
TCP window size	amount of unacknowledged data allowed in flight; limited by receiver buffer	sender throttled (no loss)
Socket buffer	OS buffer for send/receive data; limited by system memory	blocking or drop if full
IP packet size	full IP datagram (header + data); limited by MTU of path	fragmentation (IPv4) or drop (IPv6)
MTU	max IP packet size per link; limited by link technology	packet fragmented or dropped
PMTU	smallest MTU along full path; limited by weakest link	packet dropped if too large
Fragmentation	splitting IP packet into smaller pieces; limited by MTU rules	reassembly required at destination
Frame	link-layer packet carrying IP packet + CRC; limited by MTU	frame dropped



Switch table of S: 3 Entries
ARP table of A: 2 Entries
ARP table of C: 1 Entry
IP Addresses: 5, 1 per interface

DNS order: Local -> Root -> TLD -> Authoritative

Slotted ALOHA: 37%
Unslotted ALOHA: 18%

Expression	Variables
d_prop = d / s	d = length of link (m), s = prop. speed
d_trans = L / R	L = packet length (bits), R = link rate (bps)
d_e2e = N × (d_trans + d_prop + d_proc + d_queue)	N = number of links, also depends on qns
d_e2e = N × (L/R + d/s)	L = packet length, R = link rate, d = distance, s = propagation speed
Throughput = Total data / total time	end-to-end
Bottleneck Bandwidth: min(R_c, R_s)	R_c = client link rate, R_s = server link rate
Seq# = byte offset of first byte	byte offset = position in byte stream
ACK# = next expected byte	next expected byte = first missing byte
next Seq# = Seq# + N	Seq# = starting byte, N = bytes sent
Estimated RTT: RTTE = (1 − α)RTTE + α·RTT_s	α = smoothing factor (≈0.125), RTT_s = sample RTT
DevRTT = (1 − β)DevRTT + β abs(RTT_s − RTTE)	DevRTT is calculated recursively, β is the factor of new sample RTT_s
TimeoutInterval = RTTE + 4·DevRTT	RTTE = estimated RTT, DevRTT = deviation
Average Throughput for TCP: U_sender = (N · L/R) / (RTT + L/R)	N = window size, L = segment size, R = link rate, RTT = round-trip time N is small, low utilisation
Real world Throughput for TCP ≈ 1.22 · MSS / (RTT · √p)	MSS = max segment size, RTT = round-trip time, p = loss probability
Bit Trans, Time: t_bit = 1 / R	R = link rate (bps)
Stop and Wait efficiency = Time send data/ time wait for data = (L/R) / (RTT + L/R) = 1 / (1 + 2a)	L = packet length, R = link rate, RTT = round-trip time, a = d_prop / d_trans
Delay ratio: a = d_prop / d_trans	Propagation and transmission delay ratio
GBN/SR efficiency: η = 1 if N ≥ 2a+1, else N/(2a+1)	N = window size, a = delay ratio
Minimum frame size: frame size ≥ 2×d_prop×R OR 2 × d_trans ≥ RTT d_queue and d_proc are ignored	d_trans = transmission delay, d_prop = propagation delay, R = link rate, d_trans = L/R, L is framesize
Fragmentation offset: Offset = byte position / 8	byte position = location in original packet All fragments must be multiples of 8, except last fragment
K ∈ {0,…,2^m−1}	K = backoff slots, m = min(n,10), n = collision count
RSA modulus: n = p·q	p, q = prime numbers
RSA totient: z = (p−1)(q−1)	z = Euler totient
RSA key relation: e·d mod z = 1	e = public key, d = private key
RSA encrypt: c = m^e mod n	m = message, c = ciphertext
RSA decrypt: m = c^d mod n	c = ciphertext, m = message

Message breakdown through the layers:

Application → Transport (Segmentation)
- Large message of size M bytes is split into segments
- Each of the ceil(M / MSS) segments carries up to MSS (max segment size)
- MSS does not include amt needed for headers
- MSS+TCPheader+IPheader≤MTU
- TCP: 20 bytes header per segment (no options),
- UDP does not care about MSS, just sends max possible but typically <= MTU - IPHeader - UDP Header, 8 bytes header
- Transport header overhead = No. segments * header size

Transport → Network (Encapsulation)
- Each segment becomes an IP datagram
- No further splitting unless it exceeds MTU (includes header), but unlikely
- IP header = 20 bytes, header overhead = 20 * datagrams

Network → Link (Fragmentation)
- Fragments along the way if there is a link that can't handle the full datagram
- Datagram is split into fragments, Each fragment gets its own IP header
- No. fragments = ceil(datagram size / MTU - IP header)
- Each fragment adds another IP header (20 bytes)
- all fragments except last is multiple of 8

Link Layer (Framing)
- Each (fragmented or unfragmented) datagram becomes a frame
- Ethernet header: 14 bytes, CRC: 4 bytes, Total ≈ 18 bytes
- CRC is known as a trailer

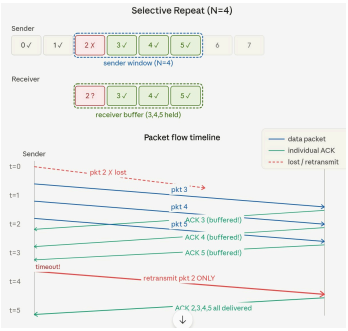
DHCP can provide multiple default gateways
- Default OS behaviour: pick first gateway in list
- gateway redundancy protocols exist (e.g. VRRP / HSRP conceptually)

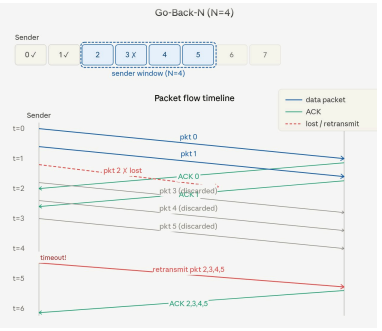
Device joins subnet:
DHCP Discover (broadcast, 0.0.0.0 → 255.255.255.255) → DHCP Offer → DHCP Request → DHCP ACK
Gets: Own IP, subnet mask, default gateway (router IP), DNS server IP

Communicating:
Compare with own subnet using dest IP AND mask

If Within subnet:
If IP not known: DNS query to DNS server → get IP
check ARP cache → if no entry: ARP request → ARP reply
send frame directly

Outside subnet:
If IP not known: DNS query to DNS server
use default gateway, check ARP cache for router → if no entry: ARP for router IP (DHCP only provides IP)
send frame to router (dst MAC = router MAC, dst IP = final destination IP)
router forwards packet





Special IPs, All 1s and all 0s is reserved for special use:
255.255.255.255 -> Limited broadcast, only local subnet
192.168.1.255 for a 192.168.1.0/24 subnet mask network -> Directed broadcast, sent to subnet with that mask
0.0.0.0 -> As dest IP, it is default route, matches any destination not in routing table, As src IP, not assigned an IP yet. Also a server listening on 0,0,0,0 means it is listening on all interfaces (ethernet, wifi, …)
192.168.1.0 for a 192.168.1.0/24 subnet mask network -> Network identifier, used to specify the network itself, not any particular device
127.0.0.1 -> Localhost, or loopback IP address, sends packets back to itself

Rdt1.0 -> Perfect channel, just send and receive
Rdt2.0 -> Error detection like checksum for bit errors + Seq number and retransmission to handle corrupted ACKs and NAKs
Rdt3.0 -> Packet loss so add timers + timeout also, seq number vry impt now

Count to infinity routing problem:
C dies, B says C = ∞, A still says "I reach C via B", B believes A, loop, cost go up very long time

RIP poison reverse:
If router X routes to destination Z via neighbour Y, then X tells Y that its cost to Z is ∞.

CS2105

Internet
- The Internet is structured as a network of networks.
- Routing in the Internet: Internet is a hierarchy of Autonomous Systems (AS)
- Hosts connect to Internet via access ISPs (Internet Service Providers), which themselves are interconnected.

Network Edge (Access Network)
- Hosts access the Internet through access network.
- Residential | institutional | mobile access networks
- access networks, physical media:wired, wireless communication links

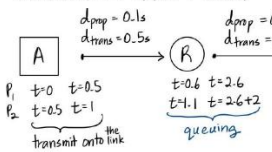
Network Core
- interconnected routers • network of networks
- *N* devices: min: *N - 1 links* • max: *N x (N-1)/2 links*

Data is transmitted through network through:

1. Circuit Switching
- End-end resources are allocated to and reserved for "call" between source & dest:
- call setup required, circuit-like (guaranteed) performance
- circuit segment idle if not used by call **(no sharing)**
- divide link bandwidth into "pieces" (reserved portion)
- frequency division | time division
- commonly used in traditional telephone network

2. Packet Switching
- Transmits packets onto the link at transmission rate R.
- link transmission rate = link capacity = **link bandwidth**
- **Benefits of message segmentation**: reduces delay, allow for retransmission of smaller parts in case of corruption, and are easier to buffer + queue behind (more fair for other packets).

Delay, Loss, Throughput
- **Loss**: Queue / Buffer ← limited capacity. Packet arriving to full queue→dropped.
- **Nodal Processing Delay**: Time required to check for bit errors and determine output link.(<msec)
- **Queueing Delay**: Time waiting in queue for transmission, depends on earlier arrived packets (network congestion level)
- **Transmission Delay**: Time to push (last bit) of packet into link $d_{trans} = \frac{L(bits)\ \text{packet length}}{R(bits/sec)\ \text{link bandwith}}$.
- **Propagation Delay**: Time to propagate to next router. $d_{prop} = \frac{d\ (m)}{s\ (m/sec)}$, d length of physical link, s propagation speed.
- End to end packet delay: total time taken for packet to travel from source to destination.
- **Throughput**: How many bits can be transmitted per unit time, measured for end-to-end communication (bandwidth ← specific link). • 1 milisecond = $10^{-3} second$
- Throughput = (the amount of data sent) / (time for transmission) $\iff L/(\text{RTT} + d_{trans})$



Internet Protocol Stack
- **Application**: Run network application | DNS | DHCP | HTTP | Security: HTTPS
- **Transport**: Process-to-Process | TCP | UDP | Security: SSL/TLS
- **Network**: host to host | IP | RIP (routing) | ICMP (x implemented by all) | Security: IPsec, VPN
- **Link**: Device to device | Ethernet | Bluetooth | WiFi | Security: IPsec, VPN

  Packet ≙ frame.
  Example: Ethernet, WiFi, PPP (Point to Point Protocol)
- **Physical**: Moves individual bits within link-layer frame from one node to another. Link and transmission medium dependent.

2. APPLICATION LAYER

Principles of Network Applications
- **Client-Server**:
  - Server waits for incoming requests and provides required services to client. Permanent IP
  - Client initiates contact with server and requests service. Do not communicate directly with each other. Might have dynamic IP (eg: browser & web server)
- **Peer-To-Peer (P2P)**:
  - No always-on server. Arbitrary end systems directly communicate (any node can act as client & server). Peers are intermittently connected and change IP addresses
  - Self-scalability. Peers request service from other peers, provide service in return to other peers.

Process Communication
- **Socket**: A software interface that process uses to send messages and receive messages from network.
- **IP Address**: 32-bit ← identify a host
- **Port Number**: A 16-bit integer ← identify a receiving process. (1 to 1023 is reserved, assigned by IANA)

Transport Layer Protocols
- **Transmission Control Protocol (TCP)**
  - Reliable data transfer. Connection-oriented service • Flow control. Congestion control: Throttle sender when network overloaded
  - Does not provide: Timing. Throughput guarantee. Security
- **User Datagram Protocol (UDP)**
  - Unreliable data transfer
  - Connectionless
  - Does not provide: reliability, flow control, congestion control, timing, throughput guarantee, security

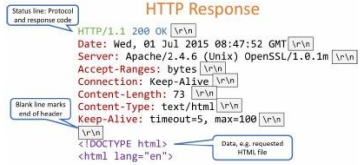
Application-Layer Protocols
An application-layer protocol defines:
- Types of messages exchanged, e.g. request and response messages.
- Message syntax: fields and how they are delineated.
- Semantics of the fields, i.e. what the information means.
- Rules for when & how to send a message and respond to messages.

Web & HTTP
- **Webpage**: Consists of base HTML files + referenced objects. Each object is addressable by URL.
- URL made up of hostname/path name. (http://www.comp.nus.edu.sg/ cs2105/img/doge.jpg)
- **HyperText Transfer Protocol** is Web's app-layer protocol.
- Client-server model: client initiates TCP connection (creates socket) to server, port 80
- **Stateless**: server maintains no information about clients, and **Reliable**: Over TCP.
- **Three-way Handshake**: Client sends small TCP segment to ask for connection, server acknowledges and responds, client acknowledges and sends it back with request message (optional data).

HTTP versions
- **RTT**: Round trip time, time taken for request/response to travel from server and back to client, does not include transmission delay. (RTT = $d_{prop} \times 2$)
- **Non-persistent HTTP**:
  Response time = $2\times$RTT + file transmission time (per object)
  1 RTT (establish connection), 1 RTT (request & get the object)
- **Persistent HTTP**:
  Same server $\iff$ same host $\iff$ same persistent connection. Subsequent HTTP sent over same TCP connection.

HTTP Request


- **HTTP 1.0 Methods**: **Sequential**. Non-persistant. GET (gets object), POST (posts form data), HEAD (gets header without body).
- **HTTP 1.1 Methods**: Support **Pipelining**, send requests back to back | n objects with 1 RTT | Persistent | GET, POST, HEAD, PUT (uploads file to path specified), DELETE
- **HTTP 2.0**: Support **multiplexing**. Response can come back in any order, even partially.

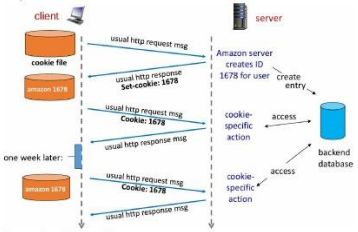
HTTP Response


Date: indicates the server response time

Last-Modified: The time the object is last modified
Content-Length: in byte
Conditional GET: In cache, specify date of cached copy in HTTP request. Server response contains no object (empty message body) if cached copy is up to date.

HTTP Status Code
200 Ok Request successful, requested object follows
304 Not Modified Object has not changed since specified date/time
301 Moved Permanently New location to follow
403 Forbidden Server declines to show the webpage
404 Not Found Requested object not found
500 Internet Server Error Unspecified error

Cookies
- HTTP is designed to be "stateless", server maintains no information about past client requests.
- **Cookie**: http messages carry "state":
  1) cookie header field of HTTP req/res messages
  2) cookie file kept on user host, managed by browser
  3) back-end database at Web site



Domain Name System
DNS translates between hostname and IP addresses. Client must carry out a DNS query to determine the IP address corresponding to the server name.
- **DNS: Resource Records (RR)**

RR Format: <name, value, type, TTL>

Type	Name	Value
A (adress)	Hostname	IP Address
NS (name server)	Domain, e.g nus.edu.sg	Hostname of authoritative name server for domain
CNAME (canonical name)	Alias for real name, e.g. www.comp.nus.edu.sg	The real name, e.g. www0.comp.nus.edu.sg
MX (mail exchange)	Domain of email address, e.g. xyz@gmail.com	Name of mail server managing the domain

- distributed database implemented in hierarchy of many name servers
- application-layer protocol: hosts, name servers communicate to resolve names (address/name translation)
- DNS runs over **UDP/53**.
  Root servers: Provides IP addresses of the TLD servers.
  Top-level domain (TLD) servers: For each top-level domain (com, org, net, country top-level domains), provides the IP addresses for authoritative DNS servers.
  Authoritative servers: Every organisation with publicly accessible hosts on the Internet
- Local DNS servers have local cache, acts as proxy, forward query into hierarchy if answer not found locally.
- **DNS Caching**: Cache mapping, which expires after some time (TTL: time to live). Cached entries may be out-of-date. (eg: If host changes IP address, it might not propagate globally until all caches expires)

DNS Name Resolution
Iterative query: Local DNS server goes back and forth with each server in the hierarchy.

Recursive query: The server that receives the query will be the one doing the next query.

Chain: Local DNS → Root DNS → TLD DNS → Authoritative DNS.

Socket Programming
Process sends/receives message through sockets (blackbox - abstraction interface between processes and transport layer protocols).

Within the same host, two processes communicate using inter-process communication (defined by OS)

UDP
Uses SOCK_DGRAM sockets.
1. (Server) Create serverSocket, port = x.
2. (Client) Create clientSocket.
3. (Client) Create datagram with server IP and port = x, and send datagram via client socket.
4. (Server) Read UDP segment from serverSocket.
5. (Server) Write reply to serverSocket, specifying client address and port number.
6. (Client) Read datagram from clientSocket.
7. (Client) Close clientSocket.

TCP
1. (Client) if the port# is not specified by socket.bind(), OS will assign a randomly chosen port
2. (Server) welcome_socket.listen(*N*), *N* = max backlog
3. client_sock.connect((*server IP, server port*))
4. ISN is randomly chosen between [0, 2^{32}-1]
5. In bi-directional communication, both directions of communication can choose same ISNs (initial sequence number)
6. Recall TCP has similar feature with GBN & SR (which handles pkt re-ordering)
7. telnet gaia.cs.umass.edu 80 ← The server should already listen to port 80 before any client connects to it | Otherwise, telnet will report error | Client cannot force server to open port

UDP vs. TCP Socket
- **UDP Socket:**Sender attach dest IP address + port # to each packet. (OS attach return info (source IP + port)). Receiver extracts source IP + port # from packet.
- **TCP Server:** Server process must first be running & have welcome socket. Contacted TCP Server forks new socket to communicate with client. • Server uses (client IP + port #) to distinguish clients.• When client creates its socket, client TCP establishes connection to server TCP.
- **TCP Client:** Creating TCP socket, specifying IP address, port number of server process • Multiple sockets for multiple connections • Every connection has its own socket instance
- **TCP vs. UDP Differences**
- In TCP, connection remains in place until one of two processes closes it. •Sending process doesn't need to attach a destination IP / port # to the bytes in sending attempt
- In UDP, Server use one socket to serve all clients. •No connection is established before sending data. •Sender explicitly attaches destination IP address and port # to each packet. •Transmitted data may be lost or received out-of-order

3. TRANSPORT

Transport and Network Layer
- **Transport** layer takes care of logical communication between **processes**.
- **Network** layer takes care of logical communication between **hosts**. (best-effort, unreliable)
- **Segment**: H_t Contains source and dest port #

H_t	msg

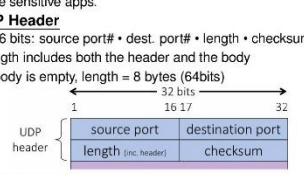
- **IP Datagram**: H_n contains source and dest IP addresses, H_t carries one transport-layer segment that contains source and dest port #.

H_n	H_t	msg

UDP
- UDP adds very little service on top of IP.
- **Multiplexing at sender**: UDP gathers data, forms packets, passes to IP.
- **De-multiplexing at receiver**: IP datagrams (from different sources) with the same dest port # will be directed to the same UDP socket
- **Unreliable**: UDP transmission used by loss tolerant and rate sensitive apps.

UDP Header
- ∀16 bits: source port# • dest. port# • length • checksum
- length includes both the header and the body

if body is empty, length = 8 bytes (64bits)



UDP Checksum
- Allows for error detection. Might fail for multi-bit error.
- (Sender) Include checksum value into UDP checksum field
- Apply binary addition on every 16-bit integer (checksum field currently 0).
- If carry from MSB, add 1 to result (wrap).
- Compute 1's complement (flip) to get UDP checksum.
- When receiving, sum segment again. All 1s if correct.

Principles of Reliable Data Transfer (rdt)
- Factors: **Packet corruption, Packet loss, Packet reordering, Packet (Long) Delay.**
- Finite State Machines to describe protocol.

Reliable Data Transfer: **Service Model**

rdt 1.0 (Perfectly Reliable)
- Assumption: Underlying channel is reliable.
- S… creates packet and sends, Receiver extracts and d… data to …

rdt 2.0 (May flip bits / Corruption)
- Assumption: Underlying channel may flip bits.
- Receiver uses checksum to detect bit errors, sends NAK if corrupted. Sender resends if NAK received.
- **Problem: If ACK or NAK corrupted**, no guaranteed way to recover. Can't detect duplicate if packet is resent.

rdt 2.1
- Add **sequence number to packet**, alternate 1 & 0. Sequence number detects duplicates.
- Same as rdt2.0, but receiver knows if it is duplicate.

rdt 2.2
- **Use ACK of last packet sequence number for NAK.**
- Receiver explicitly include seq. no, duplicate ACKs results in retransmit current pkt.

rdt 3.0 (Corruptable, Lossy, Delay)
- Assume corruption, packet loss/delay, **no re-order**.
- To detect packet loss, use **sender timeout**. Sender retransmits if no ACK received till timeout.
- If packet/ACK just delayed, retransmission may generate duplicates but receiver can use seq. no. to detect.

Remark
1. rdt 2.1-3.0, alternate ACK 1&0
2. rdt 2.0-3.0, Stop-and-wait protocol: Sender sends one packet at a time, then waits for receiver response
3. Fraction of time sender busy sending, (stop and wait) $U_{sender} = \frac{L/R}{RTT + L/R} \iff \frac{d_{trans}}{d_{trans}+RTT}$

Summary
1. **1.0** - Scenario: no error, Features: nothing
2. **2.0** - Scenario: data corruption, Features: checksum, ACK/NAK
3. **2.1** - Scenario: data + ACK/NAK corruption, Features: checksum, ACK/NAK, seq num
4. **2.2** - Scenario: Same as 2.1, Features: NAK free
5. **3.0** - Scenario: data + ACK/NAK corruption, packet loss, Features: checksum, ACK/NAK, seq num, timeout/re-transmit

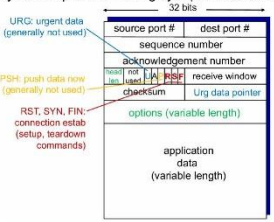
Pipelining
- **Pipelined Protocols**: sender allows multiple yet to-be-acknowledged packets.
  - range of sequence numbers must be increased
  - buffering at sender and/or receiver
  - assumption same as rdt 3.0, no re-order pkts
- Number of packets sent at once is called **window size**.
- **Benchmarked Pipelined Protocols**: Go-Back-N (GBN), Selective Repeat (SR).
- Assumption of corruption, packet loss / delay.

Go-back-N
- ACK n means all packets $\leq n$ have been received.
- Requires k bits in packet header for 2^k sequence number.
- (Sender) Timer for oldest unACKed packet
- (Sender) On timeout, retransmit all packets
- (Receiver) only accepts ACK packets that arrive in order, discards out of order packets ACK last in-order sequence # (cumulative ACK).
- simple receiver: need only remember expectedSeqNum

Selective Repeat
- Receiver needs a buffer
- Receiver individually acknowledges all correctly received packets.
- Receiver buffers out-of-order packets & send ACK
- Sender maintains timer for each unACKed packet, When timer expires, retransmit only unACKed packet.
- Upon a timeout event, retransmits 1 pckt (individual timer)

TCP
- **Connection oriented**: Exchange of control msgs inits sender, receiver state before data exchange. **Point to point**: one sender, one receiver. The connection is **duplex** (bidirectional data flow). **Reliable in-order.**
- TCP socket is identified by: (source IP addr, source port #, dest IP addr, dest port #).
- **Multiplexing**: TCP gathers data from processes, form transport-layer segments including app data and 4-tuple pass to network layer.
- **Demultiplexing**: Connection socket created, server already noted 4-tuple. Subsequent packets directed, or demultiplexed, to the appropriate socket using those 4 values.
- TCP creates **buffers** at both sides after handshaking.

TCP Segment / Header
- The maximum segment size (MSS) depends on maximum transmission unit of link-layer protocol (MTU).
- Max. file size w/o exhausting TCP seq.#: 2^{32} bytes. Take $2^{32} - 1st\ seg.\#$ to get prev. file size if wrap up
- #Packets = L / MSS, Total bytes sent = #packets* header size + file size
- MSS exclusive of header size
- TCP Seq.#: "byte#" for first byte of data in segment.
- TCP ACK.#: "seq#" of next byte expected by receiver.
- TCP only ACKs up to the missing byte (cumulative ACK)



- receive window: # bytes receiver willing to accept ← flow control
- Offset: Header size in 32-bits words (eg: 5 means header is 20 bytes long)

TCP Sender Event
1. (Sender) single retransmission timer
2. Retransmissions are triggered by:
   timeout events | duplicate ACKs
3. Retransmit only oldest unACKed packet

TCP Receiver Event
1. Arrival of In-order Segment
   i. All data already ACKed
      Delayed ACK: wait up to 500ms & send ACK if no segment arrives.
   ii. Got segment not ACKed
      Send cumulative ACK immd ← ack∀2 segments

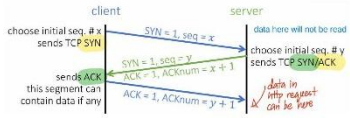
TCP Timeout Value
- Short timeout: premature timeout and unnecessary retransmissions.
- Long timeout: slow reaction to segment loss.
- Timeout interval should $>$ than RTT
- TCP takes sampleRTT (RTT_S) - once every RTT
- Estimated RTT, RTT_ϵ
- $RTT_\epsilon = (1-\alpha)\cdot \text{RTT}_\epsilon + \alpha \cdot \text{RTT}_s$

$$RTT_{\text{dev}} = (1-\beta)\cdot RTT_{\text{dev}} + \beta \cdot |RTT_s - RTT_\epsilon| \quad (1)$$

(TimeoutInterval = EstimatedRTT + 4*DevRTT)

TCP Fast Retransmission
- Timeout period is often relatively long.
- If sender receives 3 duplicate $\iff$ 4 ACKs for same segment, resend segment immediately.

TCP 3-Way Handshake / Closing



- Closing: Client, server each close their side of connection, send TCP segment with FIN bit = 1
- No more sending of data after FIN, but can still receive

Remark
To prevent duplicates from reordering:
1. a sequence# is not reused until any previously sent packets with the same sequence# are not in the network (large sequence# field)
2. IP protocol specifies TTL in pkt header (x live forever)

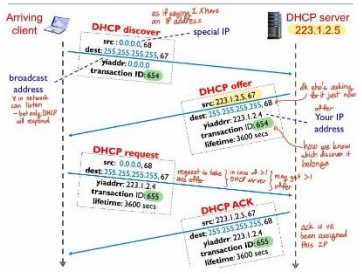
4. NETWORK

Network Layer Services
- Network layer delivers packets to receiving hosts.
- Router checks data integrity & dest. IP then forward packet to the next router
- Network layer protocols in every host and router (IP)
- **Forwarding**: Moving of incoming packet to appropriate output link.
- **Routing**: Calculation of path taken by packets from sender to receiver.

IP Address

- **IP address** 32-bit identifier associated with every interface of host or router
- Host gets an IP address either through manual configuration by sys admin, or auto assigned by a **DHCP** server.

Special Addresses	Present Use
0.0.0.0/8	Non-routable meta-address for special use
127.0.0.0/8	Loopback address. A datagram sent to an address within this block loops back inside the host.
10.0.0.0/8 172.16.0.0/12 192.168.0.0/16	Private addresses, can be used without any coordination with an Internet registry.
255.255.255.255/32	Broadcast address. All hosts on the same subnet receive a datagram with such a destination address.



DHCP: Dynamic Host Configuration Protocol

- **DHCP** allows a host to dynamically obtain its IP address from DHCP server when it joins network.
- IP address is renewable, allow reuse of addresses (only hold address while connected), support mobile users to join network.
- DHCP: 4-step process:
  1. Host broadcasts "DHCP discover" message | src: 0.0.0.0,68 | dest: 255.255.255.255, 67
  2. server responds with "DHCP offer" message
  3. Host requests IP address: "DHCP request" message
  4. DHCP server sends address: "DHCP ACK" message
- DHCP may provide host additional network information, e.g. IP of first-hop router, local DNS server, network mask.
- DHCP runs over **UDP**. DHCP server port 67, client port 68.

IP Address & Network Interface

- IP address is associated with a network interface.
- Host usually has one or two network interfaces (e.g. wired Ethernet and WiFi), A router typically has multiple interfaces.
- **IP Addr** comprises network/subnet prefix and host ID.

Subnet

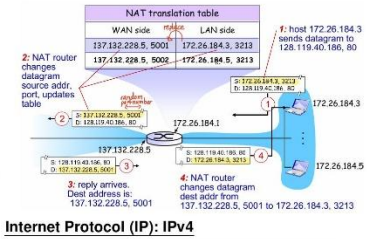
- **Subnet** is a network formed by a group of "directly" interconnected hosts.
- **Classless Inter-domain Routing** (CIDR): Internet's IP address assignment strategy.
- Address format: $a.b.c.d/x$, where x is the no. of bits in subnet prefix of IP addr. (aka match the first x bits)
- **Subnet mask** is used to determine which subnet an IP address belongs to.
- made by setting all subnet prefix bits to "1"s and host ID bits to "0"s.

IP Address Allocation

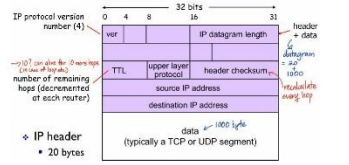
- how to get an IP address → get block from ICANN/IANA; get block from ISP
- **Hierarchical Addressing**: Allows efficient way of routing.
- **Longest Prefix Match**: Choose one with longer match.

Network Address Translation (NAT)

- Support 2^{16} simultaneous connections NAT Routers must:
- **Replace** (source IP address, port #) of every outgoing datagram to (NAT IP address, new port #), **Remember** (in NAT translation table) the mapping from (source IP address, port #) to (NAT IP address, new port #) **Replace** (NAT IP address, new port #) in destination fields of every incoming datagram with corresponding (source IP address, port #) stored in NAT translation table
- **Benefits**: Single public IP for NAT router allows multiple private IP address | Change (private IP) addr of hosts in local network without notifying outside world | We can change ISP w/o changing addresses of hosts in local network | Hosts inside local network not explicitly addressable or visible by outside world (security plus)



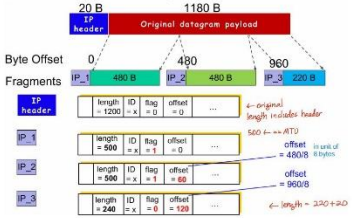
Internet Protocol (IP): IPv4



- Checksum: **Recalculate every hop**
- ver: IP protocol version number (e.g. IPv4, IPv6)
- Length: B of IP datagram + 20 bytes header.
- Identifier, flags, fragment offset: support fragmentation & reassembly
- Upper layer protocol: Only used at final dest, determine if UDP/TCP (for Internet). Checksum also uses 1s complement.

IP Fragmentation & Reassembly

- Different links, different MTU (Max Transfer Unit, inclusive of packet header).
- **Flag**(frag flag) is set to 1 if next fragment from same segment, 0 if this is the last fragment.
- Reassemble only at **final destination**
- **Offset** is in expressed in unit of 8-bytes
- Rmb to add header B to last pkt



Routing Algorithms

Path between all pairs of host needs to determined | Need to define a protocol / service guarantee | Every node/host on the internet should use the same protocol | Every router on the path should cooperate

- **Data plane**: local, per-router function, determines how datagram arriving on router input port is forwarded to router output port (pck forwarding)
- **Control plane**: network-wide logic, determines how datagram is routed among routers along endend path from source host to destination host (routing algo)
- **Intra-AS routing**: Finds good path btwn two routers within AS. Commonly used protocols: RIP, OSPF. Single admins: No policy needed
- Bellman Ford: $D_u(z) = \min_{a \in N}\{c(u,a) + D_a(z)\}$
- Computation does not include newly updated value in the same round. Only use DVs from neighbors.
- Repeat for $n-1$ times
- Shortest paths with k hops are correct

RIP (Routing Information Protocol)

- **RIP** implements the DV algorithm.
- Uses hop count as the cost metric (insensitive to network congestion).
- Exchange routing table every 30 seconds over UDP port 520.
- "Self-repair": if no update from a neighbour router for 3 minutes, assume neighbour failed.
- Iterative: The process continues until no more information is available to be exchanged between neighbors.
- Asynchronous: It does not require that all of its nodes operate in the lock step with each other.

Internet Control Message Protocol

- **ICMP**: used by hosts & routers to communicate network-level information like echo request/reply
- ICMP messages carried in IP datagrams, ICMP header starts after IP header, Located in transport layer, but used for network layer
- **ping**: check if remote host will respond to us
- **traceroute**: display route that messages take to get to a remote host | Send series of small packets across network with increasing TTL | ICMP error message sent to datagram's source address | router might not implement ICMP and might not respond to the request
- ICMP header: Type + Code(Sub type) + Checksum + others

Type	Code	Description
8	0	echo request (ping)
0	0	echo reply (ping)
3	1	dest host unreachable
3	3	dest port unreachable
11	0	TTL expired
12	0	bad IP header

4. LINK

Link Layer Services

Bit transmision time: time taken to transmit 1 bit | Source and destination MAC address changes at different points | Source and destination IP x change

- IP datagrams encapsulated in link-layer data frames
- Diff link might use diff link layer protocol
- Each link needs to be **addressed** → Framing
- Need a **protocol** → Link access control
- Need to handle **errors** → Reliability, Detection
- Physically implemented on adapter (aka Network Interface Card NIC)

Multiple Access Links and Protocols

ideal protocol

1. Collision free: Node does not receive 2+ signals at same time
2. Efficient: When 1 node wants to transmit, it can send at rate R
3. Fairness: When m nodes want to transmit, each can send at average rate of R/M
4. Fully decentralized: No special node for coordination

Random Access Protocol

Slotted ALOHA:

Designs:
All frames of equal size, L bits
Time divided into slots of equal length
Length = Time to transmit 1 data frame = L/R
Operations:
Nodes start to transmit only at the beginning of a slot
If collision, re-transmit frame in each subsequent slot with probability p until success
Possible for all nodes to **send nothing**
Efficiency: yes when 1 node. many nodes: 37%

Pure (unslotted) ALOHA

No time slot | No synchronization **Operations:**
Nodes transmit frame immediately
If collision, wait for 1 frame transmission time and retransmit frame with probability p until success
Chance of collision increases: frame sent at t_0 collides with other frames sent in (t_0-1, t_0+1)
Max efficiency: 18%

Carrier Sense Multiple Access (CSMA)

Sense the channel before transmission
Collisions may still occur due to propagation delay

CSMA/CD (Collision Detection)

If collision detected: Abort transmission
Binary Exponential backoff:
After m-th collision, choose K from $(0, 1, \ldots, 2^m - 1)$ and wait for K time units
$p = 1/2^m$ | Ethernet: 1 time unit is set to 512 bit transmission times
For both CSMA:
Discard pkt after 16 transmission attempts
Minimum Frame Size: Ethernet: 64 bytes

$$2 \max(d_{prop}) \leq d_{trans}$$

collision free: no | efficiency: yes | fair | decentralized

Taking Turn Protocol

Higher efficiency. Throughput: almost R (polling/ token overhead) | Send up to max number of frame

Polling

Taking turns | master node "invites" slave nodes to transmit in turn | polling overhead | single point of failure of master node | Decentralized: No

Token Passing

(Special frame) token is passed from one node to next sequentially | token overhead | single point of failure (lost token) | Decentralized: Yes

Channel Partitioning

TDMA (time division multiple access

Collision Free: Yes | Inefficient: Unused slot go idle | Fair | Decentralized
Data/Time frame: Collection of N time slots
Max Throughput for a node: R/N

FDMA (frequency division multiple access)

Same idea as TDMA, except dividing using frequency

Error Detection

Add Error Detection and Correction bits EDC to data | Not 100% reliable | Usually more EDC yields better detection

Single Bit Parity Checking

- 1 add. bit to make number of 1s even
- Can detect odd number of single bit errors
- Cannot detect even number of single bit error
- The probability of undetected errors in a frame approach 50%

2D Parity Checking

- Can detect and correct single bit errors
- Can detect any two-bit error

Cyclic Redundancy Check

D: Data bits (Binary) | G: Generator of r + 1 bits | R: r bit CRC | modulo 2 arithmetic | XOR
Operations:
- Append r bit 0s to D; R (r bit CRC) = remainder
- **Sender** sends (D, R)
- **Receiver** divides (D, R) by G, remainder = 0 means no corruption

Detect all odd number of single bit errors
Detect all burst errors of up to r bits
All burst errors of greater than r bits with $p = 1 - 0.5^r$
CRC aka polynomial code, $x^5 + x^4 + 1 \rightarrow 110001$

Switched Local Area Networks

Local Area Network (LAN)

LAN technologies: IBM Token Ring: IEEE 802.5 standard | Ethernet: IEEE 802.3 standard | Wi-Fi: IEEE 802.11 standard | Asynchronous Transfer Mode(ATM)

Ethernet Frame Structure

- **Preamble:** 7 bytes of 10101010 | Followed by 1 byte of 10101011 (aka "start of frame") | Used to synchronize receiver and sender clock rates (Width of bit) | E.g. How to tell between 19 or 20 zeros without knowing width of bit?
- **Type**: Higher level protocol used, usually IP. (others e.g. ARP, AppleTalk), permits Ethernet to multiplex network-layer protocols.
- **Data:** Max:1500B (Determined by link MTU);Min:46B
- **CRC:** Corrupted frame will be dropped

Ethernet Data Delivery Service

Unreliable: receiving NIC doesn't send ACK or NAK to sending NIC | CSMA/CD with binary backoff

Ethernet Switch

CSMA/CD | Switches buffer packets | Store & forward Ethernet frames | No collisions | No IP | Transparent | Must be spanning tree (No loop)

Switch Forwarding Tables

<MAC address of host, interface to reach host, TTL>

1. Record incoming link, MAC address of send. host
2. Query table using MAC destination address
3. If entry found in table:
   drop frame if the dest. = sending link; Else forwards
4. broadcast on all interfaces except arriving interface

MAC address

Every NIC has a (Media Access Control) address | 48 bits | Allocated by IEEE | First 3 bytes identifies vendor of adaptor | Broadcast address: FFFFFFFFFFFF
MAC add.: link-layer address used to move frames over single link | Permanent
IP add.: network-layer address used to move datagrams from source to dest. | Dynamic

Address Resolution Protocol (ARP)

Each IP node (Routers and hosts) has ARP table: Stores mappings of IP & MAC in the **same subnet**
<IP address, MAC address, TTL>

ARP: Sending Frame in Same Subnet

1. A knows B's MAC address (in ARP table):
   - A creates frame with B's MAC address and send.
   - Only B will process frame, all other hosts ignore.
2. A does not know B's address:
   (a) A broadcasts an ARP query packet, containing B's IP address. The destination MAC address is set to FF-FF-FF-FF-FF-FF.
   (b) All other nodes in the same subnet will receive this ARP query packet, but only B will reply it.
   (c) A caches B's IP-to-MAC address mapping in its ARP table (until TTL expires).

ARP: Sending Frame in DIFF Subnet

(A and B are in diff subnets connected by router R.)

1. A send a frame with R's MAC address as destination address (A to B IP datagram)
2. R creates frame with B's MAC address as destination address

6. NETWORK SECURITY

Some Remark

Monoalphabetic ciper: 26! possible mappings. Easily broken with statistical analysis of frequent letters.
Block Ciper: Message to encrypt processed in blocks of K bits. Possible key: $2^k!$ (with permutation)

RSA: Creating public/private key pair

1. Choose two large prime numbers p, q.
2. Compute $n = pq$, $z = (p-1)(q-1)$.
3. Choose e (with $e < n$) that has no common factors with z (e and z are relatively prime.)
4. Choose d such that $ed - 1$ is exactly divisible by z. ($ed \bmod z = 1$).
5. **public key**: n, e **private key**: n, d

RSA: Encryption, decryption

1. To encrypt message m, (note: $m < n$), compute $c = m^e \bmod n$
2. To decrypt received bit pattern c, computer $m = c^d \bmod n$

Message Integrity & Digital Signature

- Ensure message not altered without detection
- **Cryptographic Hash function**: computationally infeasible to find any two different messages x and y such that $H(x) = H(y)$ | MD5 | SHA-1 | SHA-2 | SHA-3 | Small change in input →large change in output
- Message Authentication Code: $H(m+s)$
- **Digital Signature**: Unforgeable | Verifiable | Non-repudiation

Certification Authority (CA)

- **Certificate**: Owner identity | Owner's public key | Valid time window | CA's signature
- **CA**: Verify individual/ entity identity | Validate the applicant's right to claim credentials such as domain names for server | Maintain a list of revoked certificates | Issues and signs digital certificates to websites | Maintains a directory of public keys

Firewall

Prevent Denial of Service DoS attacks | Prevent illegal access of internal data | Allow only authorized access to inside network | Access Control Lists (ACL): Rules applied top to bottom | Limitation: IP spoofing (can't know if data from claimed source)

- **Stateless packet filtering**: internal network connected to Internet via router firewall
- router filters packet-by-packet, decision to forward/drop packet based on src & dest. IP address | TCP/UDP src & dest. port num | ICMP message type | TCP ACK and SYN bits

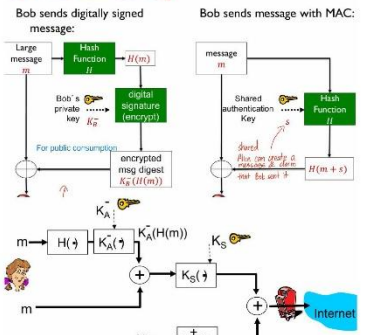
Stateless packet filtering: more examples

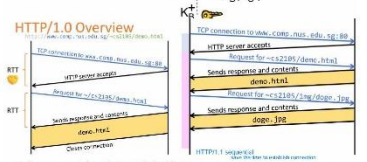
Policy	Firewall Setting
No outside Web access.	Drop all outgoing packets to any IP address, port 80
No incoming TCP connections, except those for institution's public Web server only.	Drop all incoming TCP SYN packets to any IP except 130.207.244.203, port 80
Prevent Web-radios from eating up the available bandwidth.	Drop all incoming UDP packets - except DNS and router broadcasts.
Prevent your network from being used for a smurf DoS attack.	Drop all ICMP packets going to a "broadcast" address (e.g. 130.207.255.255).
Prevent your network from being traceroute	Drop all outgoing ICMP TTL expired traffic

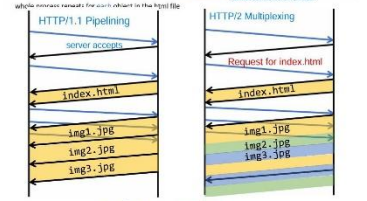
Command & Protocol

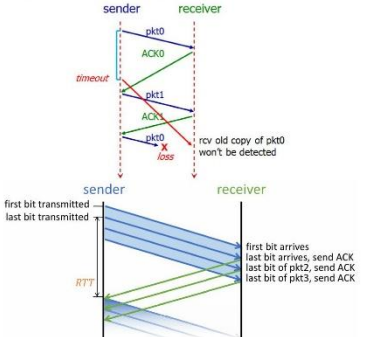
1. **ping**: ICMP | see if a remote host will respond to us | Send echo request to a specific IP and host return echo reply (w round trip time)
2. **nslookup**: find the IP corresponds to a domain, vice versa | return local DNS first (follow the chain)
3. **traceroute**: tell the domain's IP | respond time for every hop (tested 3 times) | 1 is the network gateway | ISP - global gateway - ISP - host (likely)
4. **dig**: Query DNS Servers | look at RR table
5. **telnet**: Build for interacting with remote computer | Check connectivity on certain port | telnet [domain name or ip][port] | error message: indicate closed port or remote server is not listening on that port
6. **curl**: Transfer data to or from a server | curl [website]
7. **ifconfig eth0**: MAC add. on ether card
8. **arp**: Display entries in ARP cache | _gateway = router (∀ not in routing table will be sent to here)

Digital signature vs MAC









$$U_{sender} = \frac{3 \times d_{trans}}{d_{trans} + RTT} = \frac{3 \times 0.008}{30 + 0.008} = 0.00081$$